

prompt-evolution-ga

A Deep Dive: evolving natural-language LLM prompts
with a genetic algorithm and a compile-and-run fitness function

Explainer for Salman Adnan's portfolio

What this document is

An exhaustive, beginner-safe walkthrough of the `prompt-evolution-ga` project: what it does, how every module works, what the committed results actually say, and where the code and its own documentation disagree. It assumes no prior knowledge of genetic algorithms, large language models, or C++ tooling. Every technical term is defined at first use.

This is a working document for the owner's own understanding and interview defense. It is deliberately blunt about the project's weak points, because a flaw you learn about here is a flaw you can defend in a room; one an interviewer finds first is not.

Contents

1	The project in one page	3
1.1	The three design choices that define the project	3
1.2	Provenance and honesty note	4
2	Architecture: the whole system at a glance	5
2.1	Directory structure	6
3	The evolutionary loop	7
3.1	The cycle	7
3.2	The loop in pseudocode	7
3.3	The Individual	8
4	Selection: three schemes	10
4.1	The three schemes, and what actually distinguishes them	10
4.2	Roulette selection, and its guard	10
5	The operators: crossover and mutation by LLM	12
5.1	breed(): the offspring pipeline	12
5.2	The empty-result guard, repeated four times	12
5.3	Failure-conditioned mutation: the best idea in the project	13
5.4	The output directives	14
6	Fitness: the compile-and-run pipeline	15
6.1	Sandboxing: the details that matter	15
6.2	Early stopping, and how it deflates scores	16
6.3	Parallelism	17
7	The dataset and the split	18

8	The LLM layer	20
8.1	Client construction: double-checked locking	20
8.2	The retry state machine	21
9	Cost tracking	23
10	Orchestration, checkpoints and resume	24
10.1	Pre-flight	24
10.2	Signal handling	24
10.3	Checkpointing and resume	24
10.4	The holdout evaluation	25
11	The analysis scripts	26
11.1	run_baseline_eval.py	26
11.2	visualize_test_results.py	26
12	What the committed results actually say	27
12.1	The headline: the GA lost	27
12.2	Cost, verified	27
12.3	The finding the README misses: evolution went backwards	28
12.4	The truncated winner	29
12.5	The broken genealogy	30
13	Where the documentation and the code disagree	32
13.1	The license contradiction	32
13.2	The g++ failure mode is described backwards	32
13.3	The paper's abstract claims comparisons that were never run	32
13.4	The paper describes a configuration that does not exist	33
13.5	corrections.tex: a patch file, partially applied	33
13.6	The run directory naming drifted	33
13.7	Minor	34
14	Verification record	35
14.1	Everything labelled inferred in this document	35
15	Summary: the honest scorecard	37
15.1	What is genuinely good	37
15.2	What to fix first	37

1 The project in one page

Large language models that write code are unreasonably sensitive to the wording of the instructions you give them. Telling a model “you are an expert competitive programmer, consider edge cases” can produce measurably different code from “just give me the code”. Finding the wording that works is normally a human sitting there, guessing, re-running, and guessing again.

This project asks: *can a genetic algorithm do that guessing automatically, and can we prove whether it worked?*

Jargon: Large language model (LLM)

A program trained on enormous amounts of text that, given some text, predicts plausible text to follow. Modern ones (here, Google’s Gemini) are good enough at this that they can write working computer programs from an English description of a problem. They are not databases and not calculators: the same question can yield different answers, and small changes in how you ask can change the answer a lot.

Jargon: Prompt, and system prompt

A **prompt** is the text you send an LLM. Many LLM interfaces split this in two: a **system prompt**, which sets standing instructions and persona (“you are an expert competitive programmer, always think about edge cases”), and a **user message**, which carries the actual task (“here is the problem, solve it”). This project holds the user message fixed and evolves *only the system prompt*. That is the whole point: the task never changes, so any change in code quality is attributable to the wording of the standing instructions.

Jargon: Genetic algorithm (GA)

An optimisation method that copies biological evolution. You keep a **population** of candidate solutions. Each is scored by a **fitness function**. The fitter ones are more likely to be chosen as **parents**. Parents are combined (**crossover**) and randomly tweaked (**mutation**) to make **offspring**, which form the next **generation**. Repeat. Over many generations the population drifts toward higher fitness. Nothing here understands the problem; it only knows which candidates scored better.

1.1 The three design choices that define the project

1. **The chromosome is English prose.** In a textbook GA, a candidate solution is a fixed-length vector of numbers or bits. Here a candidate is an entire natural-language instruction prompt. This choice drives everything else, and it breaks the usual GA guarantees (Section 5).
2. **Fitness is objective and brutal.** A candidate prompt is not judged by asking another model “is this a good prompt?”. It is judged by: send the prompt plus a real competitive-programming problem to Gemini, take the C++ it writes, *actually compile it* with g++, *actually run* the resulting binary against the problem’s real test cases, and score the fraction of tests that pass. There is no room to argue with a failed test.
3. **The genetic operators are themselves an LLM.** You cannot meaningfully “swap the second half” of two English paragraphs and get English. So crossover and mutation are performed by a second, stronger model (Gemini 2.5 Pro) which is asked to merge, extend, trim or rephrase the prompt text.

The one-sentence summary

A genetic algorithm whose population is English prompts, whose fitness is “does the C++ this prompt produced actually compile and pass the tests”, and whose crossover and mutation operators are a second LLM.

1.2 Provenance and honesty note

The repository contains a course paper (Computational Intelligence), co-authored with Ghufuran Alvi; the implementation is Salman Adnan’s work. Throughout this document, where a claim in the README or the paper conflicts with what the committed code and data actually show, **the code and data win**, and the conflict is called out in a red box. There are several, and two of them are substantial.

2 Architecture: the whole system at a glance

Two external systems sit outside the project's control: Google Cloud Vertex AI (which serves the Gemini models) and the local machine's C++ toolchain (`g++`). Everything else is ten Python files with no framework and no database.

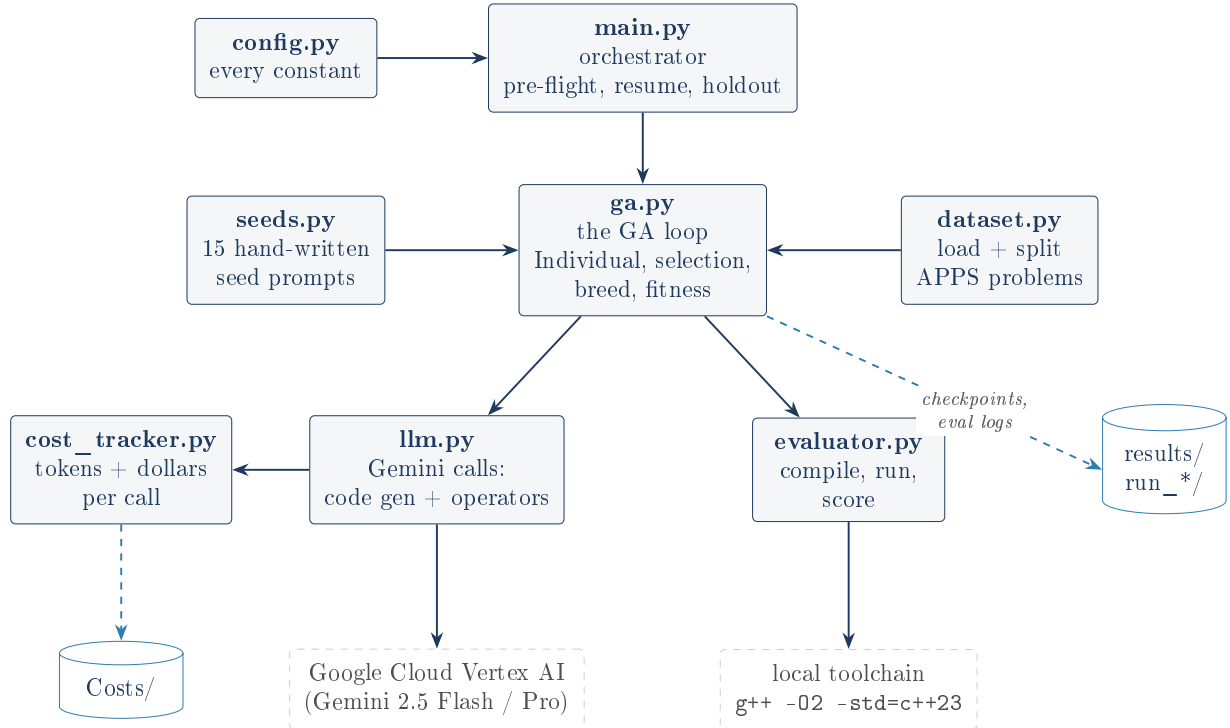


Figure 1: Module map. Solid arrows are calls; dashed arrows are writes to disk. Dashed boxes are systems outside the project: the model API and the C++ compiler. Every fitness number the project reports depends on both of them.

2.1 Directory structure

```

prompt-evolution-ga
├── main.py  entry point: pre-flight, GA, holdout eval
├── ga.py    the GA core (495 lines, the heart)
├── llm.py   Vertex AI client, code gen, operators
├── evaluator.py  compile, run, score
├── dataset.py  APPS loading and splitting
├── seeds.py   15 seed prompts (only 5 are used)
├── config.py  all hyperparameters
├── cost_tracker.py  token and dollar accounting
├── run_baseline_eval.py  evolved vs baseline on the test split
├── visualize_test_results.py  rebuilds the 5 comparison figures
├── Dataset/APPS/
│   ├── train/ 100 problems used here
│   └── test/  200 problems used here
├── results/
│   ├── run_20260429_210411/ tournament run
│   ├── run_20260430_014526/ roulette run
│   ├── run_20260430_014530/ elitism run
│   └── comparison/ the 4-way test-set comparison + figures
├── Costs/  per-run cost reports
└── corrections.tex  loose patch notes for the paper

```

Figure 2: Repository layout. Of 1,428 tracked files, 1,201 are the APPS dataset and most of the rest are run artifacts; the actual program is ten Python files totalling roughly 2,300 lines.

3 The evolutionary loop

This is the centre of the project. Everything else feeds it.

3.1 The cycle

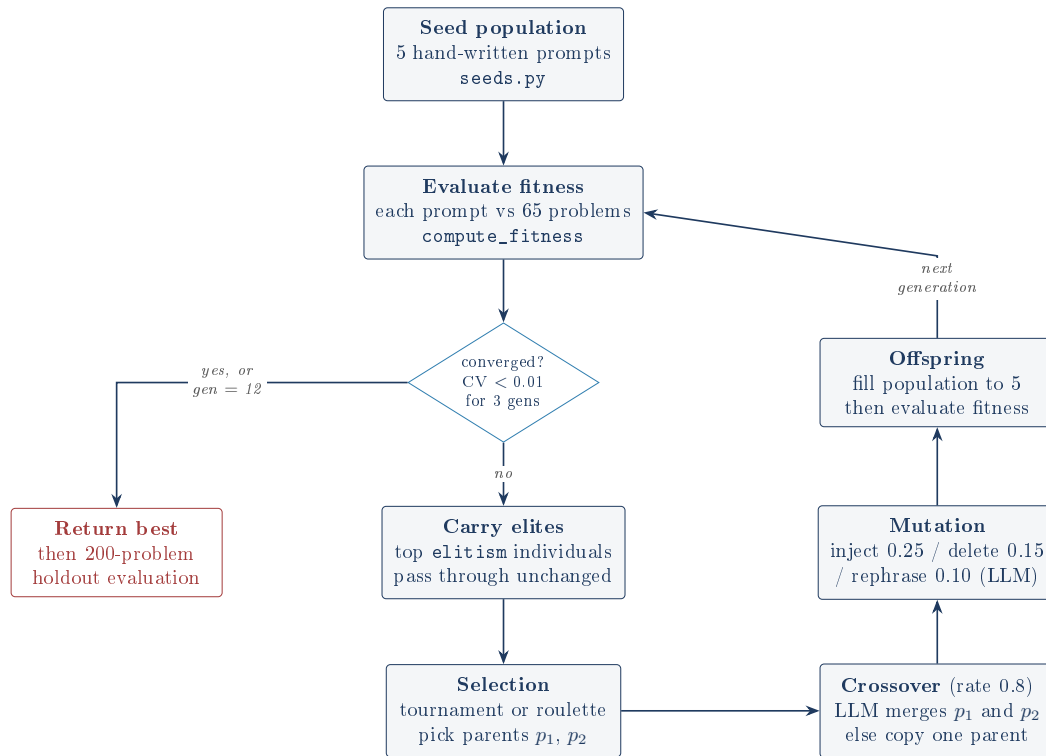


Figure 3: The generational cycle, as implemented in `ga.run`. The population is fully replaced every generation except for the elites. With the shipped settings the loop runs 12 generations over a population of 5.

3.2 The loop in pseudocode

`ga.run`: the generational loop (condensed from `ga.py`, lines 312-495)

```

input: seed_prompts (5), problems (65), scheme, elitism
output: best individual, fitness history
rng ← Random(RANDOM_SEED = 42)
select ← roulette_select if scheme = "roulette" else tournament_select
if resuming then
    population ← checkpoint.population; restore rng state
else
    population ← [Individual(p) for p in seed_prompts]
    for each ind in population:
        if a seed checkpoint exists: restore its fitness (skip re-evaluating)
        else: ind.fitness ← compute_fitness(ind, problems); write checkpoint
    stagnant ← 0
for gen = 1 to GENERATIONS (12):
    sort population by fitness, descending
    stats ← {best, mean, worst, std, cv} over the population
    append stats to history; save checkpoint of generation gen-1

    if 0 ≤ cv < 0.01 then // population has gone homogeneous

```

```

    stagnant ← stagnant + 1
    if stagnant ≥ 3: break // converged, stop early
else stagnant ← 0

new_pop ← population[0 : elitism] // elites survive untouched
off_idx ← 0
while |new_pop| < POPULATION_SIZE (5):
    p1 ← select(population, rng)
    p2 ← select(population, rng)
    allow_mut ← (off_idx ≥ CROSSOVER_ONLY_SLOTS = 2)
    new_pop.append( breed(p1, p2, rng, allow_mut) )
    off_idx ← off_idx + 1

for each child in new_pop[elitism:]: // elites keep their old score
    child.fitness ← compute_fitness(child, problems)

population ← new_pop; sort descending
append final stats to history
return population[0], history

```

The two details in that loop worth remembering

Crossover-only slots. The first two offspring of every generation (`CROSSOVER_ONLY_SLOTS = 2`) are bred with mutation switched off. This deliberately reserves part of each generation for pure recombination, so mutation cannot scramble every child.

Elites are not re-evaluated. An elite carries its old fitness number forward rather than being re-scored. Since the LLM is called at temperature 0 this is defensible, but it does mean an elite's fitness is a measurement from an earlier generation, never refreshed.

3.3 The Individual

```

1  @dataclass
2  class Individual:
3      prompt: str                # THE CHROMOSOME: an English instruction prompt
4      fitness: float = -1.0      # mean pass rate; -1.0 means "not yet evaluated"
5      uid: str = field(default_factory=lambda: uuid.uuid4().hex[:8])
6      parent_a: str = ""         # uid of parent A \
7      parent_b: str = ""         # uid of parent B  | genealogy
8      operators: str = ""        # e.g. "crossover+inject" |
9      intermediate_steps: dict = field(default_factory=dict) # / prompt after each operator
10     failure_examples: list = field(default_factory=list)    # feeds the inject mutation

```

Listing 1: ga.py, lines 49-58. The chromosome and its provenance.

Jargon: Dataclass

A Python shorthand that turns a list of named fields into a class, generating the boilerplate constructor automatically. `@dataclass` above the class is what triggers it. `field(default_factory=...)` is needed for defaults that must be freshly created per instance (a new random uid, a new empty list), rather than shared between all instances.

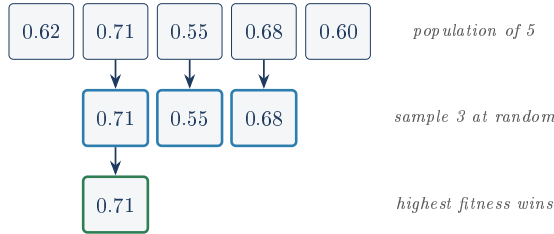
The last four fields are the project's most interesting engineering idea. Because LLM-mediated operators do not preserve structure, an offspring can silently wander away from what its parents encoded. `parent_a`, `parent_b`, `operators` and `intermediate_steps` record the full derivation of every individual, so you can reconstruct after the fact whether an operator helped or just scrambled a good prompt. `intermediate_steps` stores the prompt text after *each* operator in the chain

(`after_crossover`, `after_inject`, `after_delete`, `after_rephrase`), so a bad step can be pinpointed. Section 12.5 covers what happened when this data was actually used.

4 Selection: three schemes

Selection decides which individuals get to be parents. The project implements two mechanisms and exposes three schemes over them via `--scheme`.

Tournament selection (TOURNAMENT_SIZE = 3)



Roulette selection (fitness-proportional)

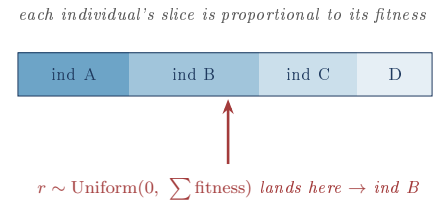


Figure 4: The two selection mechanisms. Tournament applies constant pressure regardless of the fitness *scale*; roulette's pressure collapses when everyone scores similarly, which is exactly what happens here (all fitness values sit near 0.65).

4.1 The three schemes, and what actually distinguishes them

```

1 # in ga.run:
2 scheme = selection_scheme.lower()
3 select = roulette_select if scheme == "roulette" else tournament_select
4 elitism = elitism_count if elitism_count is not None else config.ELITISM_COUNT
5
6 # in main.py, the only place elitism_count is ever passed:
7 elitism_count = 1 if args.scheme == "elitism" else None

```

Listing 2: `ga.py` lines 329-333, plus `main.py` line 357. How `--scheme` maps onto behaviour.

So the three schemes decompose into just two independent switches:

<code>--scheme</code>	selection function	elitism	note
tournament (default)	<code>tournament_select</code>	<code>config.ELITISM_COUNT = 0</code>	
roulette	<code>roulette_select</code>	<code>config.ELITISM_COUNT = 0</code>	
elitism	<code>tournament_select</code>	1 (hardcoded in <code>main.py</code>)	tournament + elitism

Table 1: The `elitism` scheme is not a third selection mechanism. It is the `tournament` scheme with one elite. The paper's own corrections file confirms this reading.

The scheme names oversell the experiment

Presented as “three selection schemes”, the experiment is really a 2×1 design: two selection mechanisms (tournament, roulette) at elitism 0, plus one extra cell (tournament at elitism 1). Roulette-with-elitism was never run. So the results cannot separate “elitism helps” from “elitism helps *tournament*”, and there is no way to tell whether the elitism run's distinct behaviour comes from the elite or from an interaction. With a population of 5 and one run per cell, no scheme comparison here can carry statistical weight; there is no repetition to average over.

4.2 Roulette selection, and its guard

```

1 def roulette_select(population, rng):
2     fits = [max(ind.fitness, 0.0) for ind in population] # clamp: -1.0 sentinel -> 0.0
3     total = sum(fits)

```

```
4     if total == 0:
5         return rng.choice(population)           # degenerate: pick uniformly
6     r = rng.uniform(0, total)
7     cum = 0.0
8     for ind, f in zip(population, fits):
9         cum += f
10        if cum >= r:
11            return ind
12    return population[-1]                       # float-rounding safety net
```

Listing 3: ga.py, lines 245-256.

Two defensive details worth noticing. The `max(ind.fitness, 0.0)` clamp matters because `-1.0` is the project’s sentinel for “infrastructure failed, this is not a real score”; without the clamp a negative would corrupt the running total and could make the cumulative sum non-monotonic. The trailing `return population[-1]` catches the case where floating-point rounding leaves `cum` a hair below `r` after the last individual. Neither is exotic, but both are the kind of thing that is quietly wrong in most first drafts.

5 The operators: crossover and mutation by LLM

Why the operators had to be an LLM

A classic GA chromosome is a fixed-length vector, so crossover is “take the first k genes from A and the rest from B”, and mutation is “flip a random bit”. Both preserve validity by construction: the result is always a well-formed chromosome.

An English paragraph has no such structure. Splicing the first half of one paragraph onto the second half of another produces something that is not a valid instruction prompt. So this project delegates the operators to Gemini 2.5 Pro and asks it, in English, to merge or edit the prompts. The result is always fluent English. It is *not* always related to its parents, and that is the trade (see the honest box at the end of this section).

5.1 breed(): the offspring pipeline

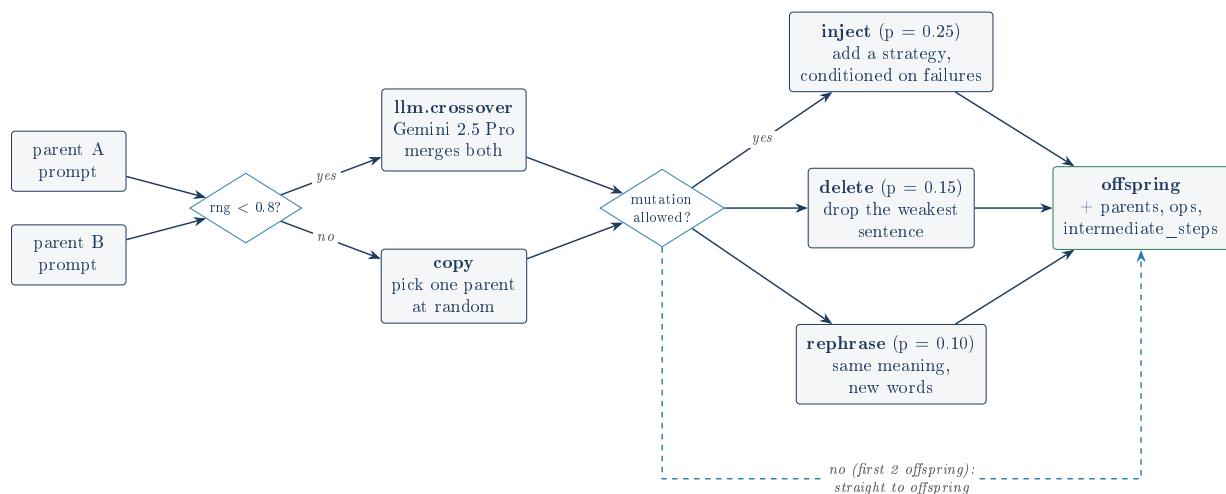


Figure 5: `ga.breed`, lines 259-309. The three mutations are independent coin flips applied in sequence, not a choice of one: an offspring can receive all three, or none.

The three mutation probabilities (0.25, 0.15, 0.10) are independent, so the chance an offspring escapes mutation entirely is $(1 - 0.25)(1 - 0.15)(1 - 0.10) \approx 0.574$, and the chance it gets all three is $0.25 \times 0.15 \times 0.10 = 0.00375$. The expected number of mutations per mutable offspring is $0.25 + 0.15 + 0.10 = 0.5$.

5.2 The empty-result guard, repeated four times

```

1  if rng.random() < config.CROSSOVER_RATE:
2      res = llm.crossover(pa.prompt, pb.prompt)
3      prompt = res if res.strip() else rng.choice([pa.prompt, pb.prompt]) # fall back
4      ops.append("crossover")
5  else:
6      prompt = rng.choice([pa.prompt, pb.prompt])
7      ops.append("copy")
8
9  steps["after_crossover"] = prompt
10
11 if allow_mutation:
12     if rng.random() < config.MUT_INJECT_PROB:
13         res = llm.mutate_inject(prompt, failure_examples=combined_fails)
14         prompt = res if res.strip() else prompt # empty -> keep previous prompt
15         ops.append("inject")

```

```

16     steps["after_inject"] = prompt
17     # ... delete and rephrase repeat the identical shape

```

Listing 4: `ga.py` lines 271-298. The pattern that keeps a failed LLM call from destroying a chromosome.

Every operator is wrapped in the same guard: if the model returns nothing usable, keep what you had. This is the right instinct. A silent empty string from an API would otherwise propagate an empty prompt into the population, and an empty prompt would score near zero and pollute the generation.

The guard checks that output exists, not that it is complete

`res.strip()` is truthy for *any* non-blank text, including text the model was cut off mid-word producing. That is not hypothetical here: it is exactly what happened, and it is documented in Section 12.4. The right check is a completeness check (does the prompt end at a sentence boundary? was the response truncated by the token cap?), and the code does not do one. Note also that `ops.append("inject")` runs *unconditionally* once the coin flip passes, even when the call came back empty and the prompt was left unchanged. So the `operators` string can claim an individual was injected when nothing was injected. Any later analysis of “which operators helped” inherits that error.

5.3 Failure-conditioned mutation: the best idea in the project

The `inject` mutation is not blind. It is fed real failures from the parents’ most recent evaluation, so the model is asked to add a strategy targeting problems the parents *actually got wrong*.

```

1  def mutate_inject(prompt: str, failure_examples: list | None = None) -> str:
2      hint = ""
3      if failure_examples:
4          lines = []
5          for f in failure_examples[:3]:
6              line = f" - Problem {f['problem_id']} (rating {f.get('rating','?')}) : reason={f['reason']}"
7              if f.get("compile_error"):
8                  line += f", compile_error={f['compile_error'][:80]!r}"
9              elif f.get("expected"):
10                 line += f", expected={f['expected'][:60]!r}, actual={f['actual'][:60]!r}"
11                 lines.append(line)
12             hint = ("Recent failure examples (address at least one):\n" + "\n".join(lines)
13                  + "\nAdd a strategy that targets these specific failure patterns.")
14
15     user = ("Add one new, specific coding strategy to this instruction prompt. "
16           "Examples: 'validate your output against the examples', "
17           "'consider time complexity', 'check for off-by-one errors'."
18           f"{hint}\n\nOutput only the modified prompt.\n\nPrompt:\n{prompt}")

```

Listing 5: `llm.py`, lines 281-303. Building the failure-conditioned mutation prompt.

Why this is a genuinely good design

This turns a blind random mutation into a directed one. A standard GA mutation has no idea *why* an individual is weak; it perturbs at random and lets selection sort it out. Here the mutation operator gets told “your parents failed problem train/2015 with a compile error, and problem test/0042 by printing 3 where 4 was expected”. That is a gradient signal smuggled into a gradient-free method, and it is the closest thing the project has to a genuinely novel idea.

The failures are chosen by `_pick_failures` (`ga.py`, lines 217-228), which sorts failures by a priority ranking (compile failures first, then total failures, then partial passes) and keeps at most

five. `breed` then merges both parents' failure lists, de-duplicating by `problem_id`, and caps at five.

`_pick_failures` has a logic bug that defeats its own diversity intent

```

1 for f in fails:
2     if f["reason"] not in seen or len(picked) < 3:    # <-- the "or" is the bug
3         picked.append(f)
4         seen.add(f["reason"])
5     if len(picked) >= 5:
6         break

```

The evident intent is “take one example per distinct failure reason, for variety”. But `or len(picked) < 3` short-circuits that: while fewer than three are picked, the condition is true *regardless* of whether the reason was already seen. Since the list is pre-sorted by reason priority, the first three picks are typically all the *same* reason (three compile failures, say). Only picks four and five apply the distinct-reason rule. The operator therefore usually sees a less varied failure sample than intended. Changing `or` to `and` would not fix it either; the loop needs restructuring. This is invisible at runtime because nothing checks it, and it degrades the project's best idea.

5.4 The output directives

Both the code generator and the operators are wrapped in blunt, almost shouted formatting directives (`_CODE_GEN_DIRECTIVE` and `_OPERATOR_DIRECTIVE`, `llm.py` lines 45-69), forbidding markdown fences, preamble and commentary. This is defensive prompt engineering: the pipeline needs raw C++ or raw prompt text, and any “Here is your solution:” preamble would be fed to the compiler.

Notice the layered defence: the directive tells the model not to emit code fences, and *then* `evaluator.extract_code` strips code fences anyway. Belt and braces, and correct: the directive reduces the failure rate, the extractor handles the residue.

The operator directives are a chromosome contaminant

`_CODE_GEN_DIRECTIVE` is appended to every candidate prompt at evaluation time, so every individual is really “evolved prompt + a fixed 700-character block of shouting”. The block is constant across all individuals, so it cannot bias the *comparison* between prompts. But it does mean the fitness landscape being searched is the landscape *given* that directive: the evolved prompts are not standalone artifacts, and quoting one in a paper without the directive misrepresents what was measured. The README does not mention this, and the paper's Table of parameters does not either.

6 Fitness: the compile-and-run pipeline

This is the part of the project that makes it defensible. Fitness is not an opinion.

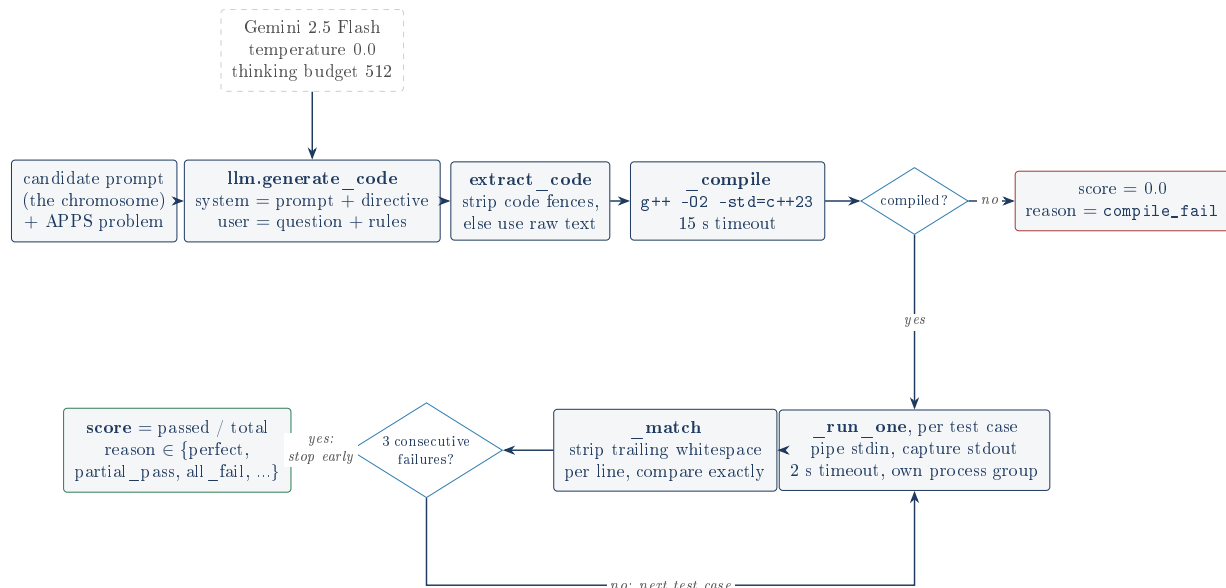


Figure 6: One fitness measurement for one prompt against one problem (`evaluator.run_tests_verbose`). An individual's fitness is the mean of this over all 65 evolution problems, computed 5-way in parallel.

6.1 Sandboxing: the details that matter

```

1 def _kill_group(proc):
2     try:
3         os.killpg(os.getpgid(proc.pid), signal.SIGKILL) # kill the whole process GROUP
4     except (ProcessLookupError, PermissionError):
5         try: proc.kill() # fall back to just the child
6         except Exception: pass
7
8 # every subprocess is launched with:
9 proc = subprocess.Popen(..., preexec_fn=os.setsid, ...) # new session => new process group

```

Listing 6: `evaluator.py`, lines 12-19 and 88-106. Killing a runaway program properly.

Jargon: Process group, and why `proc.kill()` is not enough

When a program starts other programs, they normally form a family tree. `proc.kill()` kills only the one process you launched. If the generated C++ had spawned children, or if the compiler forks (which `g++` does: it is a driver that runs `cc1plus`, `as` and `ld`), killing the parent leaves orphans running and consuming CPU forever. `os.setsid` puts each launched program into its own *process group*, and `os.killpg` kills every process in that group at once. Over three runs of roughly 13,000 model calls each producing a compile and up to a few dozen executions, that distinction is the difference between a machine that survives the night and one that does not.

Other real defences here: a 15 second compile timeout and a 2 second execution timeout (`config.COMPILE_TIMEOUT`, `config.EXEC_TIMEOUT`); `stderr` redirected to `DEVNULL` so a chatty program cannot fill a pipe buffer and deadlock; captured input/output truncated to 500 characters (`_MAX_IO_LEN`) before being logged, so a program that prints a megabyte cannot bloat the JSONL logs; and the temporary source and binary files cleaned up in a `finally` block.

This is a timeout, not a sandbox

The generated C++ is compiled and executed on the host with the user’s own privileges and no isolation: no container, no seccomp filter, no user namespace, no filesystem or network restriction. A model that emitted code calling `system("rm -rf ")`, or opening a socket, would be obeyed. The 2 second timeout bounds *how long* arbitrary code runs, not *what* it can do.

In practice the risk is low: the inputs are benign competitive-programming problems and the model has no motive. But “we ran 13,000 pieces of model-written C++ on the host, unsandboxed” is the honest description, and it is a fair thing for an interviewer to press on. The fix is well-trodden (run the binary in a container, or under `bubblewrap`, or as a nobody user in a chroot) and the code is structured such that it would be a change to `_run_one` alone.

6.2 Early stopping, and how it deflates scores

```

1  for i in range(n):
2      actual = _run_one(bin_path, inputs[i], timeout)
3      ok      = actual is not None and _match(actual, outputs[i])
4      res["tests_run"] = i + 1
5      if ok:
6          passed += 1
7          fails   = 0          # the counter RESETS on a pass: 3 CONSECUTIVE failures needed
8      else:
9          fails += 1
10         if fails >= config.EARLY_STOP_FAILURES:    # = 3
11             res["early_stopped"] = True
12             break
13
14 res["score"] = passed / n    # <-- n is the TOTAL test count, not tests_run

```

Listing 7: evaluator.py, lines 146-180. Note which denominator the score uses.

Two things are happening, and only one is obvious:

1. **Early stopping is a cost control.** Once a program has failed three tests in a row, it is almost certainly wrong, so the remaining tests are skipped. Over 65 problems $\times$ 5 individuals $\times$ 12 generations that saves a lot of wall-clock time.
2. **The score divides by the total, not by the tests actually run.** When a run stops early at test 4 of 30, the score is `passed/30`, not `passed/4`. The 27 unrun tests are silently counted as failures.

This second point is a deliberate and correct choice: assuming the unrun tests would have failed is the conservative reading, and it keeps the score comparable between an early-stopped evaluation and a complete one. It is worth being able to state clearly, because it looks like a bug and is not.

Exact-match comparison is strict, and quietly wrong for some problems

`_match` normalises by stripping trailing whitespace from each line and comparing the line lists for exact equality. That means a *correct* program is scored as failing if it prints 3.14159 where the expected file says 3.14160, or if it picks a different but equally valid answer to a problem with multiple correct outputs. APPS contains both categories.

The README argues this “deflates every prompt’s score by roughly the same amount”. That is probably true on average and is a reasonable defence, *but it is not free*: it also injects noise into the fitness landscape the GA is trying to climb, and with a population of 5 there is little averaging to damp it. A per-problem checker (float tolerance, answer-set comparison) is the

correct fix and the README says so.

A smaller note: the README refers to this logic as “`_normalise/_compare` in `evaluator.py`, line 41”. There is no function called `_compare`; it is `_match`, at line 40. Minor, but it is the kind of thing an interviewer checks.

6.3 Parallelism

```

1 with ThreadPoolExecutor(max_workers=config.PARALLEL_EVALS) as executor:    # = 5
2     futures = {executor.submit(_eval_one_problem, gen, ind, prob, i, len(problems)): i
3                 for i, prob in enumerate(problems)}
4     for future in as_completed(futures):
5         idx = futures[future]
6         try:
7             score, f = future.result()
8             scores[idx] = score
9             if f is not None:
10                with lock: fails.append(f)
11        except Exception as exc:
12            _log(f"ind={ind.uid} prob={idx+1} UNEXPECTED ERROR: {exc}", level="ERROR")
13            scores[idx] = -1.0

```

Listing 8: `ga.py`, lines 189-208. Five problems evaluated at once.

Jargon: Thread pool, and why threads are the right choice here

A **thread** lets one program do several things at once. Python’s threads are limited by the Global Interpreter Lock, so they cannot speed up pure Python computation. But they are perfectly good at *waiting*, and waiting is all this workload does: waiting for a network reply from Gemini, and waiting for `g++` and the test binary (both separate processes). `ThreadPoolExecutor(max_workers=5)` keeps five such waits in flight. `as_completed` yields futures as they finish, in completion order, which is why the code keys a dictionary from future back to index rather than relying on order.

The pool size of 5 is a rate-limit compromise, not a hardware one. There is also a deliberate jitter, `time.sleep(random.uniform(0, 1))` at the top of every evaluation (`ga.py` line 167) and `uniform(0, 2)` in the holdout path, to stop five threads from hitting the API in the same instant at the start of each batch.

The exception handler writes `-1.0`, the “this is not a real score” sentinel, rather than `0.0`. That distinction is carried carefully: `-1.0` means infrastructure broke, `0.0` means the model genuinely produced code that failed everything.

The sentinel is honest inside a generation and lost at the boundary

`compute_fitness` computes `sum(scores) / len(scores)` over the raw list, so a `-1.0` sentinel is averaged in *as a negative number*. An individual that hit one infrastructure failure across 65 problems is therefore penalised by roughly $1/65 \approx 0.0154$ of fitness more than an individual whose code merely failed every test on that problem. It is a small effect, and arguably an acceptable one (it makes flaky individuals slightly less attractive), but it is not what a reader would assume, and the distinction the `-1.0` sentinel exists to preserve is silently discarded at exactly the point it mattered. The generation-level statistics are more careful: they filter to `valid = [f for f in fits if f >= 0.0]` before computing standard deviation. So the same sentinel is treated two different ways within thirty lines of each other.

7 The dataset and the split

Jargon: APPS, and Codeforces ratings

APPS (Hendrycks et al., 2021) is a benchmark of programming problems scraped from sites like Codeforces, each with a problem statement, a set of test inputs and their expected outputs. **Codeforces rating** is a difficulty number: 800 is beginner-level, 1900 is genuinely hard. The project's subset spans 800 to 1900.

`dataset.py` does not scan directories. It hardcodes an explicit registry (`_SELECTED`, lines 9-58) of exactly which problem folders to load, grouped by split and rating. Verified by running it: 300 problems load, 100 from `train/` and 200 from `test/`.

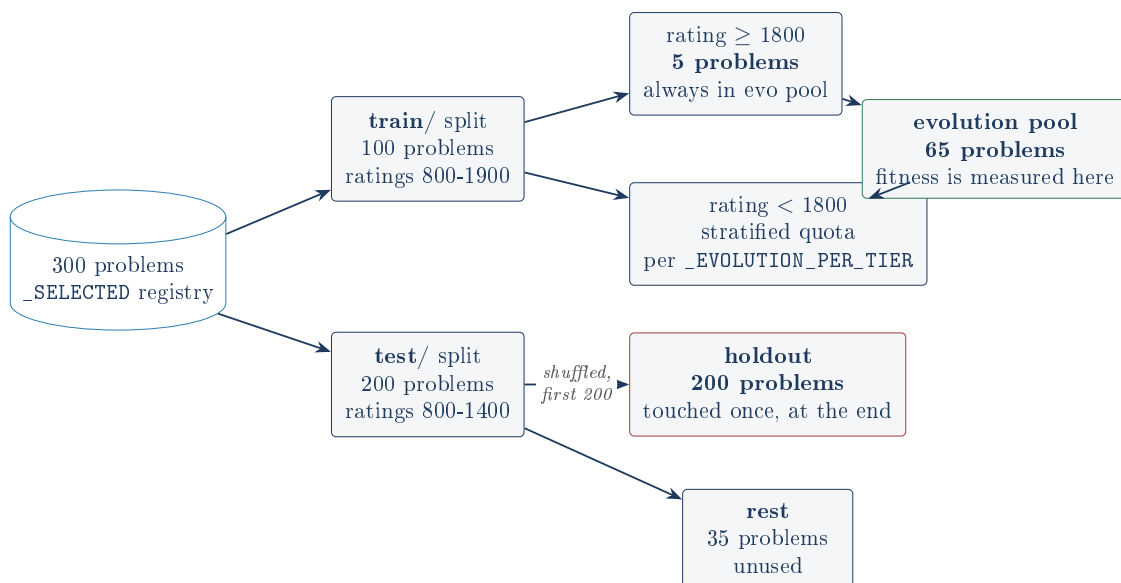


Figure 7: The split, verified by running `dataset.split`: evolution 65, holdout 200, rest 35. The five hardest train problems are forced into the evolution pool; the remaining 60 slots are filled by a per-rating quota.

Why the evolution pool and the holdout must be different problems

If you tune a prompt on the same problems you then score it on, a high score proves nothing: the prompt may simply have absorbed the quirks of those 65 problems. This is **overfitting**. The 200-problem holdout is never seen during evolution and is evaluated exactly once, at the very end, on the single winning prompt. That is the number that means something. The split is reproducible (`RANDOM_SEED = 42`) and, better, the actual chosen IDs are written to `evolution_ids.json` in the run directory, so a resumed run restores the *exact same* split from disk rather than trusting the seed to regenerate it. That is the more robust choice: it survives a change to the splitting code.

The stratification is deliberate. `_EVOLUTION_PER_TIER` (lines 61-71) fixes a quota per rating tier (7 at rating 800, 6 at 1000, and so on), so the evolution pool cannot end up accidentally all-easy. The five rating-1800-and-above problems bypass the quota entirely and are always included: with only five hard problems available, leaving their inclusion to chance would risk a pool with no hard problems at all.

Three real problems with the dataset layer

The holdout is drawn only from test/, the evolution pool only from train/. Look at the `_SELECTED` registry: `train/` spans ratings 800 to 1900, `test/` spans only 800 to 1400. So the evolution pool contains problems rated 1500, 1600, 1700, 1800 and 1900 that have *no counterpart* in the holdout, and the holdout’s difficulty distribution is materially easier than the pool the prompts were tuned on. The generalisation claim is therefore “tuned on 800-1900, tested on 800-1400”. That is not wrong, but it is not the like-for-like comparison a reader assumes, and neither the README nor the paper mentions it.

`reserved_train_problems` (lines 163-179) is dead code. It is never called from anywhere in the repository. It computes a “reserved” set of train problems using `_RESERVED_PER_TIER`, a table that exists solely to feed this unused function. It appears to be a leftover from a planned third split.

`sys.set_int_max_str_digits(1_000_000)` is called at the top of `load_problems` (line 87) with no comment. It raises Python 3.11’s guard against converting enormous integers to strings, which exists to prevent a denial-of-service. It is presumably needed because some APPS problem has a huge numeric literal in its test data, but nothing records that, and a reader cannot tell whether it is load-bearing or superstition.

8 The LLM layer

llm.py is the only file that talks to Google Cloud. It handles authentication, retries, token accounting, and the four prompt-shaped functions the GA calls (`generate_code`, `crossover`, `mutate_inject`, `mutate_delete`, `mutate_rephrase`).

8.1 Client construction: double-checked locking

```
1 _api_client = None
2 _api_lock   = threading.Lock()
3
4 def _get_client():
5     global _api_client
6     if _api_client is not None:      # fast path: no lock taken once initialised
7         return _api_client
8     with _api_lock:
9         if _api_client is not None: # re-check INSIDE the lock
10            return _api_client
11        ...
12        _api_client = genai.Client(vertexai=True, project=project,
13                                   location=config.GCP_REGION, credentials=creds,
14                                   http_options=_gt.HttpOptions(timeout=120_000))
15    return _api_client
```

Listing 9: llm.py, lines 87-126, condensed. One client, shared by five threads.

Jargon: Double-checked locking

Five threads may call `_get_client` simultaneously at startup. Without a lock, all five could see `None` and each build a client. Taking a lock on *every* call would serialise the fast path forever after. So: check without the lock (cheap, and correct once initialised), and only if it looks uninitialised take the lock and check *again* inside it, because another thread may have won the race while you waited. This idiom is famously subtle in some languages; in CPython it is safe here because assignment to a module global is atomic.

8.2 The retry state machine

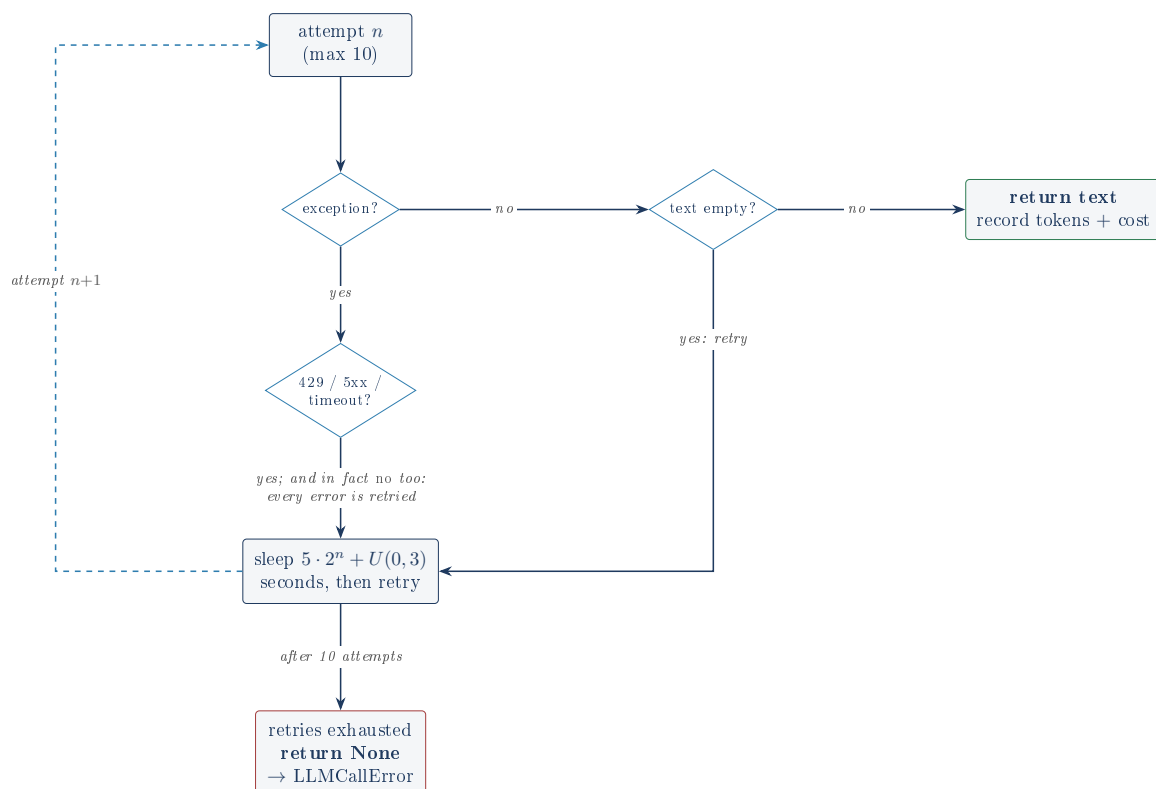


Figure 8: `llm._call_api`, lines 142-220. Exponential backoff with jitter, up to 10 attempts, covering both exceptions and the empty-response case.

Jargon: Exponential backoff with jitter

When a server says “too many requests”, retrying immediately makes it worse. So wait, and double the wait each time: 5s, 10s, 20s, 40s and so on ($5 \cdot 2^n$). The random extra (`random.uniform(0, 3)`, the *jitter*) stops five threads that were rate-limited at the same instant from retrying at the same instant and colliding again. Without jitter, parallel clients synchronise into waves.

The empty-response retry deserves note: a Gemini call can succeed at the HTTP level and return no text (the model spent its whole token budget on internal reasoning, or a safety filter fired). The code treats that as retryable, which is right. Critically, it records the cost *before* checking whether the text is empty (lines 178-188), so an empty response still gets billed and still gets counted. That is the honest accounting choice: Google charges for it regardless.

The backoff can wait 85 minutes, and every error is retried

At attempt 9 the wait is $5 \cdot 2^9 = 2560$ seconds, over 42 minutes; the cumulative worst case across 10 attempts is roughly 85 minutes for a *single* call, with no cap. Worse, the final `else` branch (lines 214-217) retries even errors that were explicitly classified as neither rate-limit, server error, nor timeout: an authentication failure or a malformed request will be retried ten times with full backoff before failing. A capped backoff (`min(wait, 60)`) and a non-retryable error class are both one-line changes.

The configured temperatures are dead constants

`config.py` defines `TARGET_TEMPERATURE = 0.0` and `OPTIMIZER_TEMPERATURE = 0.7`. Verified by grep: **neither is read anywhere in the codebase**. `_call_api` hardcodes the same values inline:

```
1 is_fit = (phase == "fitness")
2 config=_gt.GenerateContentConfig(
3     temperature=0.0 if is_fit else 0.7,                # not config.
4     TARGET_TEMPERATURE
5     thinking_config=_gt.ThinkingConfig(thinking_budget=512 if is_fit else 2048),
6 )
```

The values agree today, so nothing is broken. But editing `config.py` to change a temperature would have *no effect*, silently, which is the worst kind of configuration bug. The thinking budgets (512 and 2048) are hardcoded with no config constant at all.

Two pricing tables, one imported and never used

`llm.py` line 10 reads from `cost_tracker` import `tracker`, `PRICING`, and then never uses `PRICING`. Instead it defines its *own* `_pricing` dict at line 129 with overlapping but not identical contents, and `_cost` uses that one. So the project carries two pricing tables that can silently drift apart. Since `llm.py` always passes an explicit `cost_usd` into `tracker.record`, `cost_tracker`'s own `PRICING` is only reachable through the `CallRecord` properties when `cost_usd` is `None`, which never happens in this codebase. The whole `PRICING` table plus the three `CallRecord` cost properties are effectively dead.

`run_fitness_batch` is a stub

`llm.py` lines 242-243 define `run_fitness_batch(requests)` whose entire body raises `NotImplementedError("Batch API not used: realtime only.")`. Nothing calls it. It is a fossil of an abandoned plan to use Vertex AI's batch endpoint, which would have been roughly half the price. Similarly `evaluator.run_tests` (line 191) is a thin wrapper around `run_tests_verbose` that nothing calls.

9 Cost tracking

Why a course project needed a cost subsystem

Every fitness evaluation is a paid API call. One run is roughly 4,000 calls over 4 to 6 hours, unattended. Without live accounting you start a run, go to sleep, and find out what it cost from the bill. `cost_tracker.py` writes an updated JSON report after *every single call*, so a run can be watched from another terminal and killed if it drifts. The three committed runs cost \$5.99, \$6.16 and \$4.90, all verified from the committed reports in `Costs/`.

The tracker is a single global instance (`tracker = CostTracker()`, line 191) with a lock around all mutation, accumulating totals by phase (`fitness`, `crossover`, `mutate_inject`, ...) and by model. It writes three artifacts: an append-only `calls.jsonl` with one line per call, a live summary at `costs_live.json` in the run directory, and a second live copy under `Costs/`.

`restore_from_jsonl` (lines 77-112) is what makes `--resume` honest about money: it replays the entire `calls.jsonl` to rebuild the running total, so a resumed run reports cumulative cost across both halves rather than restarting the counter at zero.

One subtlety handled correctly, in `llm.py` lines 172-175:

```
1 in_tok = getattr(usage, "prompt_token_count", 0) or 0
2 out_tok = getattr(usage, "candidates_token_count", 0) or 0
3 think = getattr(usage, "thoughts_token_count", 0) or 0
4 out_tok += think      # thinking tokens are billed at the OUTPUT rate
```

Jargon: Thinking tokens

Gemini 2.5 models can do hidden internal reasoning before answering. Those tokens are not in the visible response but they are billed, at the output rate. Folding `think` into `out_tok` before costing is the difference between a cost estimate that matches the bill and one that badly understates it, especially for the operator calls which are given a 2048-token thinking budget. The `or 0` guards against the API returning `None` rather than omitting the field, which `getattr`'s default alone would not catch.

Three blemishes on an otherwise solid subsystem

Stale pricing rows. `PRICING` still carries `claude-haiku-4-5@20251001`, `claude-haiku-4-5-20251001` and `claude-sonnet-4-6` rows from an earlier iteration that used a different backend. The project has used only Gemini for the whole of its committed history. The README already admits this one.

A write to disk on every call, twice. `record()` calls `_write_summary` for *both* live paths while holding the lock, so every one of roughly 4,000 calls per run rewrites two JSON files synchronously with the global lock held. It is harmless at this scale (the calls take seconds, the write takes microseconds) but it is a real serialisation point, and it means a crash mid-write can leave a truncated JSON file.

Rounding accumulates inconsistently. Per-bucket totals are re-rounded to 6 decimal places on every update (`round(bucket[key]["total_cost_usd"] + call_cost, 6)`) while the grand total is not rounded until it is written. The two can therefore disagree in the last decimal. Immaterial in dollars; the kind of thing worth noticing.

10 Orchestration, checkpoints and resume

`main.py` is the entry point. It runs pre-flight checks, builds the split, wires the loggers, runs the GA, then evaluates the winner on the holdout.

10.1 Pre-flight

The pre-flight sequence (lines 200-225) checks, in order: the service-account file exists; `google-genai` and `google-auth` import; the service account parses and its `project_id` and `client_email` are printed; both APPS directories exist; the problems load; and enough problems survive to fill both splits. Each failure exits with a specific message. For a program that is about to spend six hours and six dollars, failing in the first two seconds when the credential is missing is exactly right.

10.2 Signal handling

```

1 def _on_signal(signum, frame):
2     print(f"\n[MAIN] signal {signum}: flushing costs and exiting ...", flush=True)
3     tracker.flush()
4     sys.exit(1)
5
6 signal.signal(signal.SIGTERM, _on_signal)
7 signal.signal(signal.SIGINT, _on_signal)

```

Listing 10: `main.py`, lines 337-343. Ctrl-C does not lose the cost record.

The signal handler cannot do what it looks like it does

It flushes the cost report, which is genuinely useful. But it does *not* save the current population, so pressing Ctrl-C mid-generation discards up to a full generation of work: potentially an hour of wall clock and a dollar of API spend. The resume path can only restart from the last completed generation's checkpoint.

There is also a subtler issue: `sys.exit()` in a signal handler raises `SystemExit` in the *main* thread only. With a `ThreadPoolExecutor` whose worker threads are mid-call, the interpreter will wait on the pool's non-daemon threads at shutdown, so Ctrl-C can appear to hang for up to the 120 second API timeout. Not a correctness bug, but it explains behaviour that would otherwise look like a freeze.

10.3 Checkpointing and resume

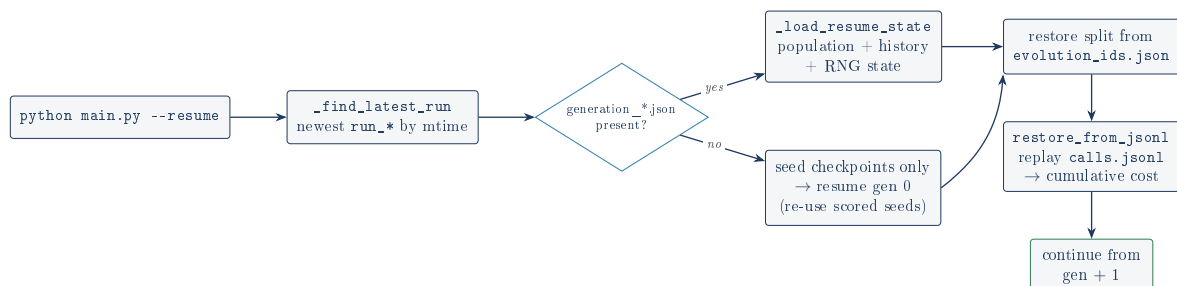


Figure 9: The resume path. Three separate pieces of state are restored: the population, the random number generator, and the cumulative cost. Restoring the RNG is what makes a resumed run take the same decisions the uninterrupted run would have.

Why the RNG state is checkpointed, and why that is unusually careful

Seeding a generator with `Random(42)` makes a run reproducible *from the start*. It does nothing for a run resumed at generation 7: a fresh `Random(42)` would replay the coin flips from generation 1, so the resumed run would take different decisions than the uninterrupted one would have.

So `ga.rng_state_to_dict` serialises Python's full Mersenne Twister internal state (a 625-element tuple) into JSON, and `rng_state_from_dict` restores it. The resumed run's next coin flip is precisely the one the original run would have made. Most student projects do not do this, and most would not notice it was missing.

Seed evaluation gets its own finer-grained checkpointing: each of the five seed prompts is scored and immediately written to `seed_nn_uid.json` (`ga.py`, lines 378-387). Generation 0 is the most expensive single generation, since all five individuals are new; a crash during it would otherwise waste the lot. On resume, already-scored seeds are restored and skipped.

The final generation is never checkpointed, and it breaks two things

`save_checkpoint` is only ever called from `_on_gen`, which `ga.run` invokes at the *top* of the loop, recording generation `gen-1`. When the loop finishes its last iteration, it exits and returns the winner without ever writing the final population.

This is not a theoretical concern. Verified against all three committed runs:

run	checkpoints	gens in <code>history.json</code>	winner in a checkpoint?
<code>run_20260429_210411</code> (tourn.)	12 (00 to 11)	13 (0 to 12)	no
<code>run_20260430_014526</code> (roulette)	12 (00 to 11)	13 (0 to 12)	no
<code>run_20260430_014530</code> (elitism)	12 (00 to 11)	13 (0 to 12)	yes

Two concrete consequences:

- 1. Resume silently rewinds.** A `--resume` after a completed 12-generation run restarts from generation 11, discarding generation 12 entirely.
- 2. It is the root cause of the broken genealogy** (Section 12.5). The winning individual of the tournament and roulette runs exists only in `summary.json` and `best_prompt.txt`; it is in no checkpoint, so its ancestry cannot be traced. The elitism run escapes only by luck: its winner (`822eea31`) was an elite from generation 6, carried forward, and so it appears in earlier checkpoints.

The fix is one line: call `on_generation` once more after the loop.

10.4 The holdout evaluation

After the GA returns, `evaluate_on_holdout` (lines 154-175) scores the single winning prompt against all 200 holdout problems, in the same 5-way thread pool. Note the difference from `compute_fitness`: here an infrastructure failure is clamped to 0.0 (`scores[idx] = entry["score"] if entry["score"] >= 0.0 else 0.0`), rather than averaged in as -1.0. That is the third distinct treatment of the sentinel in the codebase, and this one is the most defensible.

11 The analysis scripts

11.1 run_baseline_eval.py

Scores the tournament run's evolved prompt against the strongest hand-written seed (SEED_PROMPTS[0]) on all 200 test-split problems, writing the results into `comparison.json`. It has a nice resumability property: it reads any existing JSONL first, keys by `problem_id`, and only evaluates what is missing, so an interrupted comparison can be restarted without re-paying for completed problems.

The script cannot reproduce three of the four result sets it is credited with

The constants at the top are hardcoded:

```
1 BEST_RUN_DIR    = config.RESULTS_DIR / "run_20260429_210411"      # the tournament run
   only
2 EVOLVED_JSONL   = OUT_DIR / "evolved_tournament_test_evaluations.jsonl"
3 EVOLVED_PROMPT  = (BEST_RUN_DIR / "best_prompt.txt").read_text().strip() # at IMPORT
   time
```

So the committed script produces exactly two files: `evolved_tournament_...` and `baseline_...`. But `results/comparison/` also contains `evolved_roulette_test_evaluations.jsonl` and `evolved_elitism_test_evaluations.jsonl`, and `visualize_test_results.py` *requires* all four and exits if any is missing.

The roulette and elitism evaluation logs were therefore produced by hand-editing these constants between runs, and **that variant of the script was never committed**. The headline four-way comparison in the README and the paper cannot be regenerated from this repository without editing source. There is no scheme parameter and no CLI flag; adding one is a fifteen-minute change that would close the gap.

Second, smaller: `EVOLVED_PROMPT` is read at *module import time*, not inside `main()`. Importing this module anywhere without that exact run directory present raises immediately, before any error handling can produce a useful message.

comparison.json and the figures compute the mean two different ways

`build_comparison` computes `mean_score` over *valid* entries only (`[r["score"] for r in results if r.get("score", -1) >= 0]`, divided by `len(valid)`). `visualize_test_results.py` divides by a hardcoded 200 (`sum(v["score"] for v in t.values()) / 200`), which would count a -1.0 sentinel as a *negative*.

The two agree today, and I verified why: all four evaluation logs contain 200 entries with zero infrastructure failures, so `len(valid) == 200` and the sentinel never appears. The agreement is luck, not design. A single `infra_failure` in any log would make the JSON and the figures disagree, and the figures would be wrong.

11.2 visualize_test_results.py

Regenerates the five comparison figures from the committed JSONL logs. It needs no API access, and I confirmed it works: running it reproduced all five PNGs and printed exactly the numbers the README claims (Section 12). It opens with `matplotlib.use("Agg")` before importing `pyplot`, selecting the non-interactive backend so it runs on a headless machine, and asserts up front that all four logs have 200 entries with matching problem IDs. Those assertions are the right instinct: a silent mismatch between prompt runs would produce a plausible-looking and completely wrong chart.

12 What the committed results actually say

Everything in this section was re-derived from the committed artifacts, not taken from the README.

12.1 The headline: the GA lost

Prompt	Mean pass rate (200 test problems)	vs baseline
Baseline (best hand-written seed)	0.7900	
Tournament evolved	0.7857	−0.0043
Roulette evolved	0.7717	−0.0183
Elitism evolved	0.7713	−0.0187

Table 2: Verified by recomputing from the four `*_test_evaluations.jsonl` files, and independently reproduced by re-running `visualize_test_results.py`. All four logs contain exactly 200 entries and zero infrastructure failures. **No evolved prompt beat the hand-written baseline.**

Per-problem, tournament vs baseline: **26 wins, 22 losses, 152 ties** (verified by recomputing the deltas in `comparison.json`). Tournament was the only scheme with a positive head-to-head record, and 152 ties out of 200 tells its own story: on three quarters of the problems the prompt made no difference whatsoever.

The outcome distributions are strikingly flat. Baseline: 142 perfect, 44 partial, 12 compile failures, 2 total failures. Tournament evolved: 142 perfect, 44 partial, 10 compile failures, 4 total failures. Evolution moved two problems from “compile failure” to “fails all tests”. That is the entire measured effect of the whole exercise on this metric.

The honest headline, and why it is still worth defending

The project’s result is **negative**: a genetic algorithm, given \$17 and 13,000 model calls, did not find an instruction prompt better than the one a human wrote by hand in an afternoon. That is a real, publishable finding, and the machinery that produced it is sound. The value of the project is that it is *capable of proving its own hypothesis wrong*, which most prompt-optimisation work is not, because most of it has no objective fitness function and no held-out test set. Defend the method, report the result.

12.2 Cost, verified

Run	LLM calls	Cost (USD)
run_20260429_210411 (tournament)	4,734	\$5.99
run_20260430_014526 (roulette)	4,529	\$6.16
run_20260430_014530 (elitism)	3,739	\$4.90
Total	13,002	\$17.05

Table 3: Read directly from `Costs/cost_run_*.json`. This matches the README’s “roughly 13,000 LLM calls and about \$17” exactly. Note the roulette run cost *more* than the tournament run despite 205 fewer calls: roulette’s operator calls to the expensive Gemini 2.5 Pro must have run longer or produced more thinking tokens.

12.3 The finding the README misses: evolution went backwards

This is the most important thing in this document, and neither the README nor the paper says it. It is derived entirely from the committed `history.json` of each run.

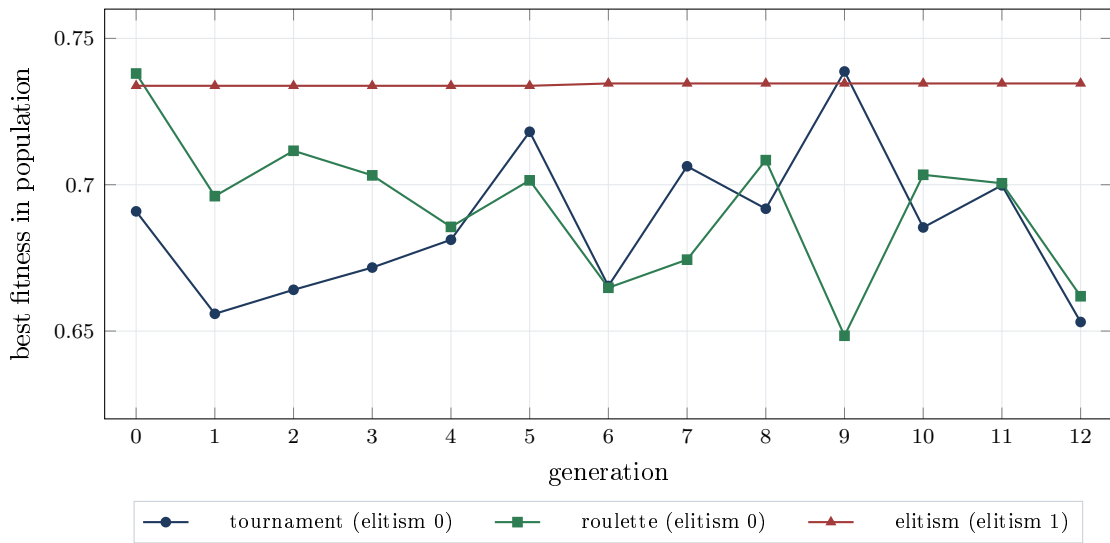


Figure 10: Best fitness per generation, plotted from the committed `history.json` of each run. The tournament and roulette curves are *not* monotonic: they wander, and both **end below where they started**. Only the elitism run is non-decreasing, which is exactly what an elite guarantees.

The README’s “evolution reliably improved fitness” is not what the data shows

The README states: “evolution reliably improved fitness on the 65-problem training pool (best-of-run fitness 0.65 to 0.73 depending on scheme)”. Those three numbers are the *final generation’s* best. Compare them with generation 0, the hand-written seeds:

run	gen 0 best (seeds)	gen 12 best (final)	change
tournament	0.6909	0.6531	−0.0378 (worse)
roulette	0.7380	0.6619	−0.0761 (worse)
elitism	0.7338	0.7346	+0.0008 (flat)

In two of three runs the GA ended with a worse prompt than the seeds it started from. The third improved by 0.0008, which is noise. The roulette run’s peak fitness was its generation 0. The tournament run peaked at 0.7387 at generation 9 and then *lost that individual*, finishing 0.086 below its own peak.

The mechanism is not mysterious and is worth being able to explain: with `elitism = 0`, the entire population is replaced every generation, so nothing protects the best individual. A good prompt found at generation 9 is gone by generation 10 unless it happens to be selected *and* survives crossover and mutation intact. With a population of 5 and an LLM rewriting the text at every step, it usually does not.

The paper’s claim (line 516) that “on the 65-problem training subset, the GA consistently produced prompts that outperformed the best” seed is contradicted by the repository’s own `history.json`. So is the Discussion’s claim (line 505) that “tournament selection with elitism delivered the most reliable training improvement due to its strict, non-decreasing best-fitness guarantee”: that guarantee is real, but the run labelled *tournament* had elitism 0 and is visibly not non-decreasing.

This is the single most important thing to be ready to discuss. The defensible position is a good one: “the run that protected its best individual held its ground; the two

that did not, regressed. That is textbook GA behaviour and my data shows it cleanly. My write-up reported the final generation's best as though it were the run's achievement, and that was wrong."

The recorded elitism_count is wrong in all three runs

main.py writes the run's config.json using the *module constant*:

```
1 (run_dir / "config.json").write_text(json.dumps({
2     ...
3     "elitism_count": config.ELITISM_COUNT,      # NOT the value actually passed to ga.run
```

but the value actually used comes from main's own argument: `elitism_count = 1` if `args.scheme == "elitism"` else `None`. So:

run	scheme	config.json records	actually used
run_20260429_210411	(none: predates --scheme)	2	0 (inferred)
run_20260430_014526	roulette	0	0
run_20260430_014530	elitism	0	1

The elitism run's own config file says it used no elitism. The tournament run's says it used *two* elites.

The "0 (inferred)" entry needs stating precisely. What is *verified*: that run's `config.json` records `elitism_count: 2`, and its `summary.json` records `selection_scheme: null`, meaning it predates the `--scheme` flag. What is *inferred*: elitism 2 cannot have been in force, because an elite mathematically guarantees best fitness never decreases, and that run's best fitness decreases repeatedly (Figure above). So either `config.ELITISM_COUNT` was 0 at run time and the 2 was written by a later edit, or the code changed mid-run. All three runs record `resumed: true`, so a code change between the original run and its resume is a live possibility.

The artifacts do not record enough to settle it.

The lesson generalises: a run directory should record the values a run *actually used*, resolved at call time, not a snapshot of module constants taken before the arguments were applied. As it stands the committed `config.json` files cannot be trusted, and the provenance of the flagship tournament run cannot be fully reconstructed.

12.4 The truncated winner

- 1 First, approach the problem systematically: analyze the input-output relationship to
- 2 identify the core algorithm, paying close attention to variable types, edge

Listing 11: `results/run_20260429_210411/best_prompt.txt`, in its entirety (160 characters).

It stops mid-sentence, at "edge". Presumably "edge cases". This is the prompt that won the tournament run and that `run_baseline_eval.py` scored at 0.7857 against the baseline's 0.7900.

The cause is `API_FALLBACK_MAX_TOKENS_OPS = 700` in `config.py`, capping every operator's output. The model was cut off mid-word, `res.strip()` was truthy, and the fragment entered the population. All three winners show it:

run	length	ends with
tournament	160 chars	"...variable types, edge"
roulette	120 chars	"...first perform a"
elitism	132 chars	"...include main(),"

Every single winning prompt is a truncated fragment

This is worse than the README’s framing of it as a one-run artifact. **All three** runs’ winners are cut off mid-sentence, and all three are between 120 and 160 characters: far shorter than the roughly 500-character seeds they descend from.

That reframes the whole result. The comparison is not “evolved prompt vs hand-written prompt”. It is “a truncated 160-character fragment vs a complete 500-character hand-written prompt”, and the fragment lost by 0.0043. One could argue the GA was doing something interesting (short prompts scoring within half a point of a careful long one is not nothing), but what it *was not* doing is the experiment as described.

The 700-token cap should never have bound at all: these prompts are 30 to 40 tokens. The likely explanation, and this is **inferred**, not verified, is that `max_output_tokens` is being consumed by Gemini 2.5 Pro’s 2048-token thinking budget against a 700-token output ceiling, so the visible output is truncated after the model has already spent its allowance reasoning. That is consistent with the thinking budget being set to 2048 for operator calls while the output cap is 700, but I could not verify it without API access.

Either way the defect is the missing completeness check flagged in Section 5: an operator’s output should be rejected if it does not end at a sentence boundary, or if the API reports a length-based finish reason. The API’s `finish_reason` field is available on the response and the code never looks at it.

12.5 The broken genealogy

The README highlights “full genealogy logging (every individual records its parents, operators, and intermediate prompt states)” as a headline engineering feature, and the paper’s abstract goes further: “Genealogy analysis reveals which instructional strategies, edge-case emphasis, pseudocode planning, output formatting, survive selection pressure and generalize across problem distributions.”

The underlying data capture is real: `parent_a`, `parent_b`, `operators` and `intermediate_steps` are populated and are written into every `generation_NN.json`. But the analysis built on it did not work:

```
1 {
2   "uid": "efd4a762",
3   "note": "not found in checkpoints"
4 }
```

Listing 12: `results/run_20260429_210411/genealogy_tree.json`, in its entirety.

```
1 Best uid : efd4a762
2 Lineage depth (parent_a chain): 0 steps
3 Total individuals in registry: 60
```

Listing 13: The corresponding `genealogy_report.txt`, excerpted.

The genealogy analysis found nothing, and the script that produced it is not in the repo

Two separate problems.

It failed. The tournament and roulette reports both show “Lineage depth: 0 steps” and a tree of “not found in checkpoints”. Only the elitism run produced a real lineage (depth 2). The root cause is established in the checkpointing box above: the final generation is never written to disk, so the winners of the two elitism-free runs exist in no checkpoint and their ancestry cannot be traced. The feature is not conceptually broken; it is defeated by a one-line omission elsewhere.

The script does not exist. Verified by `grep`: **no Python file in the repository contains**

the string “genealogy”. `genealogy_report.txt` and `genealogy_tree.json` are committed *outputs* of a script that was never committed. So the repository’s headline analysis feature cannot be re-run, inspected, or fixed by anyone reading it, including its author in six months. Consequently the paper’s abstract claim that genealogy analysis “reveals which instructional strategies survive selection pressure” is not supported by anything in this repository. The committed genealogy analysis reveals a depth of 0 for the flagship run. The raw data to do the analysis properly is present in the checkpoints; the analysis is not.

13 Where the documentation and the code disagree

Collected in one place, because these are the questions an interviewer asks.

13.1 The license contradiction

The README says MIT. The LICENSE file is not MIT.

README line 79: “Code is MIT licensed (see LICENSE).”

LICENSE line 3: “# PolyForm Strict License 1.0.0”.

These are close to opposites. MIT permits reuse, modification and redistribution; PolyForm Strict 1.0.0 permits *viewing only* and forbids any use, reuse or redistribution. The git history explains how it happened: commit 38c7f8b, “Switch license to PolyForm Strict 1.0.0 (public to view, no reuse or redistribution)”, changed the file and never updated the README sentence pointing at it.

This is the most consequential documentation bug in the repository, because it is the one with legal meaning: a reader who trusts the README could reasonably believe they had been granted MIT rights that the actual license withholds. It is a one-line fix and it should be made before anyone else reads the repo.

13.2 The g++ failure mode is described backwards

The README says it fails loudly. It scores zero silently.

README line 83 (Challenges): “no g++ on PATH means no fitness, and the fitness path fails with a plain ‘g++ not found’ rather than silently scoring zero.”

Verified on this machine, which has gcc-13 but no g++:

```
1 >>> evaluator.run_tests_verbose('int main(){return 0;}', ['1'], ['1'], 2)
2 reason: compile_fail | compile error: 'g++ not found on PATH' | score: 0.0
```

It returns `score = 0.0` and `reason = "compile_fail"`, and the GA carries on perfectly happily. The string “g++ not found on PATH” appears only inside the per-problem JSON log, indistinguishable at the fitness level from a genuine compile error. A run on a machine without g++ would complete all 12 generations, spend the full \$6 on API calls, assign every individual a fitness of exactly 0.0, and report a winner chosen at random. **The README describes the opposite of what the code does.**

Note that `RUNNING.md` gets this right: its failure table says “g++ not found on PATH in results → Install g++; every candidate scores 0 without it”. So the repository contradicts itself, and the accurate description is in the less prominent file.

The fix belongs in `main.py`’s pre-flight, which already checks the credential and the dataset but not the compiler: a `shutil.which("g++")` check, or a trial compile of a hello-world, would turn a six-hour \$6 no-op into a two-second error.

13.3 The paper’s abstract claims comparisons that were never run

Chain-of-thought and random search do not exist in this codebase

The paper’s abstract states: “Preliminary results demonstrate that evolved prompts outperform hand-crafted baselines, chain-of-thought prompts, and random search, achieving meaningful improvements in pass@1 scores.”

Three claims, and the committed repository supports none of them:

- “outperform hand-crafted baselines”: contradicted. Evolved 0.7857 vs baseline 0.7900.

The README already flags this one honestly.

- **“chain-of-thought prompts”**: verified by grep, **there is no chain-of-thought comparison anywhere in the codebase**. No script, no result file, no mention.
- **“random search”**: likewise, **no random-search baseline exists**. It is not implemented and no artifact records one.

The only comparison the repository can support is evolved vs one hand-written seed, and it went the wrong way. The word “preliminary” does some work, but not this much. If the paper is ever shown to anyone, this abstract is the first thing they read and the easiest thing to falsify.

13.4 The paper describes a configuration that does not exist

The paper (Experimental Configurations) describes a “full-scale setup: population size 15, training dataset 100 problems, 20 generations” alongside the “optimised setup” of population 5 / 65 problems / 12 generations that the code actually implements.

There is no committed artifact from a population-15, 20-generation run, and `config.py` has no such setting. One tantalising trace: `seeds.py` contains exactly **15** seed prompts, of which `main.py` uses only the first five (`SEED_PROMPTS[:config.POPULATION_SIZE]`). The remaining ten are almost certainly the population-15 setup’s seeds, left in place when the configuration shrank. This is **inferred** from the coincidence of the number 15 and the slicing, not confirmed anywhere in the source.

Note this also makes the README’s “starts from 5 hand-written seeds in `seeds.py`” imprecise: there are 15 in the file, and 5 are used.

13.5 `corrections.tex`: a patch file, partially applied

`corrections.tex` is a loose file at the repo root containing blocks of LaTeX marked “REPLACE: ... (was lines 263-277)” to be pasted into the paper. Checking whether they landed: the “Elitism (top-1)” correction *is* in the paper (line 272), so the file was partially applied. The survival-scheme wording differs from the corrected version, and the Table 2 corrections (adding the Optimizer LLM row and the temperatures) are not present in the paper’s parameter table.

So `corrections.tex` is a half-applied patch left in the repository root with no indication of which parts landed. It should either be applied and deleted, or moved somewhere that marks it as history. As a public-facing artifact it reads as an unfinished edit.

13.6 The run directory naming drifted

Both the code and `RUNNING.md` say a run directory carries its scheme in the name. The committed ones do not:

```
1 main.py line 292 creates:  run_{timestamp}_{args.scheme}
2 RUNNING.md documents:    results/run_{timestamp}_{scheme}/
3
4 but the three committed run directories are bare timestamps:
5 results/run_20260429_210411      (no scheme suffix)
6 results/run_20260430_014526      (no scheme suffix)
7 results/run_20260430_014530      (no scheme suffix)
```

The committed runs therefore predate the current naming code, which is consistent with the tournament run’s `summary.json` recording `selection_scheme: null`. Harmless in itself, but it is direct evidence that **the committed results were not produced by the committed code**, which is the thing to be careful about: every number in this repository comes from a version of the program that no longer exists in it.

13.7 Minor

`main.py` line 222 prints “loading 305 pre-selected Codeforces problems” and then loads 300 (verified). The README already notes this one.

14 Verification record

What was actually executed while writing this document, on a machine with Python 3.11+, matplotlib and numpy, but **no g++** and **no GCP credentials**:

Check	Result
All 10 modules byte-compile <code>dataset.load_problems()</code> <code>dataset.split(...)</code> <code>visualize_test_results.py</code>	Pass Loads 300 problems (100 train, 200 test), confirming the README's "305" print statement is wrong evolution 65, holdout 200, rest 35: matches config exactly Regenerates all 5 PNGs; prints tournament 0.7857, roulette 0.7717, elitism 0.7713, baseline 0.7900, matching the README table exactly
Recomputed means from the 4 JSONL logs	Independently reproduces the same four numbers; all logs have 200 entries, 0 infra failures
Recomputed win/loss from <code>comparison.json</code> Cost totals from <code>Costs/cost_run_*.json</code>	26 wins, 22 losses, 152 ties: matches the README 13,002 calls, \$17.05: matches "roughly 13,000" and "about \$17"
<code>evaluator.run_tests_verbose</code> without g++	Returns <code>score = 0.0, reason = compile_fail: contradicts</code> the README's "rather than silently scoring zero"
Best fitness monotonicity per <code>history.json</code> Winner present in any checkpoint <code>grep</code> for genealogy in <code>*.py</code> <code>grep</code> for chain-of-thought / random search <code>grep</code> for TARGET_TEMPERATURE usage README license claim vs LICENSE file	tournament and roulette non-monotonic and end below gen 0 ; elitism monotonic tournament no , roulette no , elitism yes No match : the script is not committed No match : neither baseline exists Defined in <code>config.py</code> , read nowhere README says MIT; file is PolyForm Strict 1.0.0
Not verified	The full evolution loop, every Vertex AI call, and the compile-and-run fitness path. These need a billed GCP project and a g++ install. All fitness numbers in this document are read from committed artifacts, not reproduced.

Table 4: Verification record. The repository's own analysis pipeline reproduces its headline numbers exactly from the committed logs, which is a genuine strength: the results are auditable without spending a dollar.

14.1 Everything labelled inferred in this document

Stated plainly, so nothing here is mistaken for a confirmed fact:

1. **That the tournament run used elitism 0** despite its `config.json` recording 2. Inferred from the mathematical impossibility of an elite coexisting with decreasing best fitness. The artifacts do not record the value actually used.
2. **That the 700-token operator cap interacts with the 2048-token thinking budget** to cause truncation. Consistent with the settings and with all three winners being truncated, but unverifiable without API access.
3. **That `seeds.py`'s ten unused prompts are leftovers from the paper's population-15 "full-scale setup"**. Inferred from the coincidence of the number 15.
4. **That `sys.set_int_max_str_digits` is needed for a specific APPS problem's large numeric data**. Nothing records why it is there.

-
5. **That the roulette/elitism comparison logs were made by hand-editing `run_baseline_eval.py`.** Inferred: the logs exist, the committed script cannot produce them, and no other script does.

15 Summary: the honest scorecard

15.1 What is genuinely good

- **The fitness function.** Compile-and-run against real tests is the right answer and the hard one. It is what makes the negative result trustworthy rather than arguable.
- **Failure-conditioned mutation.** Feeding real compile errors and wrong outputs into the mutation operator is a genuinely good idea: a directed mutation in a gradient-free method.
- **RNG state checkpointing.** Serialising the full Mersenne Twister state so `--resume` is decision-identical is unusually careful.
- **Process-group killing.** `os.setsid` plus `os.killpg` is the correct way to bound untrusted subprocesses, and most first attempts get it wrong.
- **Cost accounting, including thinking tokens.** The \$17 figure is real and auditable, and folding thinking tokens in at the output rate is the detail that makes it match the bill.
- **Auditable results.** The analysis pipeline reproduces the headline numbers exactly from committed logs, offline. I checked; it does.
- **A README that already admits the headline negative result** and several of its own bugs. That is rarer than it should be.

15.2 What to fix first

Ordered by how much damage each does if someone else finds it first:

1. **The license contradiction** (README says MIT, file is PolyForm Strict). One line. It is the only defect here with legal meaning.
2. **The “evolution reliably improved fitness” claim** in the README, and the paper’s matching claim. The data shows two of three runs *regressing* below their seeds. Section 12.3.
3. **The paper’s abstract**, which claims comparisons against chain-of-thought and random search that do not exist in the codebase.
4. **Checkpoint the final generation** (one line). It silently breaks resume and it is the root cause of the broken genealogy.
5. **Validate operator output for completeness.** All three winning prompts are truncated mid-sentence. Check the API’s `finish_reason`.
6. **Add g++ to the pre-flight check**, and fix the README sentence that describes this failure mode backwards.
7. **Record the elitism value actually used** in each run’s `config.json`, not the module constant.
8. **Commit the genealogy script**, or delete its orphaned outputs and drop the claim.
9. **Parameterise `run_baseline_eval.py` by scheme**, so the four-way comparison is reproducible without editing source.
10. **Statistical testing.** A 0.0043 gap over 200 problems needs a paired test or a bootstrap before it means anything in either direction. The README already says this.
11. **Delete the dead code:** `run_fitness_batch`, `run_tests`, `reserved_train_problems`, the unused `PRICING` import, the Claude pricing rows, the dead temperature constants.

The line to take in an interview

Lead with the method, not the result. The project built an objective, expensive, honest measuring instrument for a question that is usually answered by vibes, and then reported that the answer was no. The instrument works: I re-ran its analysis and it reproduces its own numbers exactly.

Then get ahead of the two things a careful reader will find: the winning prompts are all truncated fragments, and two of three runs ended below their starting seeds because nothing protected the best individual. Both have clean, short explanations. Both are more interesting than the headline. Neither is in the write-up yet, and fixing that is the next job.